



ESCUELA POLITÉCNICA NACIONAL

Escuela de Formación de Tecnólogos



TEMA:

CAJA REGISTRADORA

Sistema CRUD con Login

Asignatura: Algoritmos y estructuras de datos

Proyecto: Sistema CRUD con Interfaz Gráfica (Tkinter)

Lenguaje: Python

Integrantes:

Sebastián Caiza

Ismael Sánchez

Fabián Collaguazo

Mateo Andramunio

Dereck Ortiz

Matias Borja

Docente:

Byron Loarte Gustavo Cajamarca

Curso:

GR2

1. Introducción

El presente documento describe el desarrollo, funcionamiento y posibles mejoras del proyecto **Caja Registradora con Sistema CRUD y Login**, implementado en el lenguaje de programación **Python**, utilizando la librería gráfica **Tkinter**. El sistema permite gestionar pedidos mediante las operaciones básicas **Crear, Leer, Actualizar y Eliminar (CRUD)**, además de incorporar un **sistema de registro e inicio de sesión de usuarios**.

El proyecto fue desarrollado con fines académicos y aplica conceptos fundamentales de **programación estructurada**, manejo de **listas, tuplas, validaciones, interfaces gráficas**, y **control de flujo**, ejecutándose en un entorno de escritorio.

2. Objetivos

2.1 Objetivo General

Desarrollar un sistema CRUD en Python con interfaz gráfica que permita administrar pedidos de una caja registradora de manera eficiente, incluyendo un sistema básico de autenticación de usuarios.

2.2 Objetivos Específicos

- Implementar una interfaz gráfica utilizando Tkinter.
 - Aplicar las operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre los pedidos.
 - Utilizar listas y tuplas para el almacenamiento de información.
 - Implementar un sistema de registro e inicio de sesión.
 - Realizar validaciones de datos ingresados por el usuario.
 - Facilitar futuras mejoras y ampliaciones del sistema.
-

3. Alcance del Proyecto

El sistema permite:

- Registrar usuarios con nombre de usuario y contraseña.
- Iniciar y cerrar sesión.
- Registrar pedidos de clientes.
- Mostrar todos los pedidos registrados.
- Actualizar pedidos existentes mediante un índice.
- Eliminar pedidos del sistema.

- Calcular precios con y sin IVA.
-

4. Tecnologías Utilizadas

- **Lenguaje de programación:** Python
 - **Interfaz gráfica:** Tkinter
 - **Tipo de aplicación:** Escritorio
 - **Librerías utilizadas:**
 - **tkinter**
 - **tkinter.messagebox**
 - **random**
-

5. Estructura del Sistema

5.1 Estructura de Datos de Pedidos

El sistema utiliza una **tupla de listas**, convertida en lista (`mostrarLista`), para almacenar la información de los pedidos:

- Lista 0: Nombre del cliente
- Lista 1: Apellido del cliente
- Lista 2: Teléfono
- Lista 3: Dirección
- Lista 4: Precio sin IVA
- Lista 5: Precio con IVA
- Lista 6: Código aleatorio del pedido

Cada posición del índice representa un pedido completo.

5.2 Estructura de Usuarios

Los usuarios se almacenan en una lista que contiene:

- Lista 0: Nombres de usuario
 - Lista 1: Contraseñas
-

6. Funcionalidades del CRUD

6.1 Crear (Create)

Permite registrar un nuevo pedido ingresando los datos del cliente y seleccionando un paquete. El sistema valida los campos, calcula el precio con IVA y genera un código aleatorio para el pedido.

```
def registrar():
    nombre = entry_nombre.get()
    apellido = entry_apellido.get()
    telefono = entry_telefono.get()
    direccion = entry_direccion.get()----|
    opcion = paquete_var.get()

    if nombre == "" or apellido == "" or telefono == "" or direccion == "":
        messagebox.showerror("Error", "Completa todos los campos")
        return

    if telefono.isdigit() == False:
        messagebox.showerror("Error", "Teléfono solo debe tener números")
        return

    mostrarLista[0].append(nombre)
    mostrarLista[1].append(apellido)
    mostrarLista[2].append(int(telefono))
    mostrarLista[3].append(direccion)

    if opcion == "PC + Monitor":
        precio = 500
    elif opcion == "PC + Monitor 4K":
        precio = 2000
    elif opcion == "Laptop Pro IA":
        precio = 1500
    else:
        precio = 3000

    mostrarLista[4].append(precio)
    mostrarLista[5].append(precio * 1.15)
    mostrarLista[6].append(random.randint(1, 1000))

    messagebox.showinfo("Registro", "Registro guardado con éxito")

    entry_nombre.delete(0, tk.END)
    entry_apellido.delete(0, tk.END)
    entry_telefono.delete(0, tk.END)
    entry_direccion.delete(0, tk.END)
```

6.2 Leer (Read)

Muestra todos los pedidos registrados mediante una ventana emergente, detallando la información completa de cada pedido.

```
def mostrar_pedidos():
    if len(mostrarLista[0]) == 0:
        messagebox.showinfo("Pedidos", "No existen pedidos registrados")
        return

    texto = ""

    for i in range(len(mostrarLista[0])):
        texto += f"""
Pedido #{i + 1}
Índice: {i} (usa este número para actualizar/eliminar)
Código: {mostrarLista[6][i]}
Nombre: {mostrarLista[0][i]} {mostrarLista[1][i]}
Teléfono: {mostrarLista[2][i]}
Dirección: {mostrarLista[3][i]}
Total sin IVA: ${mostrarLista[4][i]}
Total con IVA: ${mostrarLista[5][i]:.2f}
Paquete: {paquete_var.get() if False else ""}
-----
"""

    messagebox.showinfo("Lista de pedidos", texto)
```

6.3 Actualizar (Update)

Permite modificar un pedido existente usando el índice como referencia. Se validan los datos ingresados antes de realizar la actualización.

```
def actualizar_pedido():
    if len(mostrarLista[0]) == 0:
        messagebox.showinfo("Actualizar", "No existen pedidos registrados")
        return

    indice = entry_indice.get()

    if indice == "":
        messagebox.showerror("Error", "Ingresa el índice del pedido")
        return

    if indice.isdigit() == False:
        messagebox.showerror("Error", "El índice debe ser un número")
        return

    indice = int(indice)

    if indice < 0 or indice >= len(mostrarLista[0]):
        messagebox.showerror("Error", "Índice fuera de rango")
        return

    nombre = entry_nombre.get()
    apellido = entry_apellido.get()
    telefono = entry_telefono.get()
    direccion = entry_direccion.get()
    opcion = paquete_var.get()

    if nombre == "" or apellido == "" or telefono == "" or direccion == "":
        messagebox.showerror("Error", "Completa todos los campos para actualizar")
        return

    if telefono.isdigit() == False:
        messagebox.showerror("Error", "Teléfono solo debe tener números")
        return

    mostrarLista[0][indice] = nombre
    mostrarLista[1][indice] = apellido
    mostrarLista[2][indice] = int(telefono)
    mostrarLista[3][indice] = direccion

    if opcion == "PC + Monitor":
        precio = 500
    elif opcion == "PC + Monitor 4K":
        precio = 2000
    elif opcion == "Laptop Pro IA":
        precio = 1500
    else:
        precio = 3000

    mostrarLista[4][indice] = precio
    mostrarLista[5][indice] = precio * 1.15

    messagebox.showinfo("Actualizar", "Pedido actualizado con éxito")

    entry_indice.delete(0, tk.END)
    entry_nombre.delete(0, tk.END)
    entry_apellido.delete(0, tk.END)
    entry_telefono.delete(0, tk.END)
    entry_direccion.delete(0, tk.END)
```

6.4 Eliminar (Delete)

Elimina un pedido del sistema utilizando su índice. El sistema solicita confirmación antes de eliminar el registro.

```
def eliminar_pedido():
    if len(mostrarLista[0]) == 0:
        messagebox.showinfo("Eliminar", "No existen pedidos registrados")
        return

    indice = entry_indice.get()

    if indice == "":
        messagebox.showerror("Error", "Ingresa el indice del pedido")
        return

    if indice.isdigit() == False:
        messagebox.showerror("Error", "El indice debe ser un número")
        return

    indice = int(indice)

    if indice < 0 or indice >= len(mostrarLista[0]):
        messagebox.showerror("Error", "Índice fuera de rango")
        return

    if messagebox.askyesno("Confirmar", f"¿Eliminar el pedido con índice {indice}?") == False:
        return

    mostrarLista[0].pop(indice)
    mostrarLista[1].pop(indice)
    mostrarLista[2].pop(indice)
    mostrarLista[3].pop(indice)
    mostrarLista[4].pop(indice)
    mostrarLista[5].pop(indice)
    mostrarLista[6].pop(indice)

    messagebox.showinfo("Eliminar", "Pedido eliminado con éxito")
    entry_indice.delete(0, tk.END)
```

7. Sistema de Login

El sistema incluye:

- Registro de nuevos usuarios.
- Validación de usuario y contraseña.
- Control de acceso al módulo CRUD.
- Cierre de sesión.

8. Flujo del Programa

1. Mostrar pantalla de login y registro.
2. Registrar usuario o iniciar sesión.
3. Acceder al módulo CRUD.
4. Registrar, mostrar, actualizar o eliminar pedidos.
5. Cerrar sesión y retornar al login.

9. Validaciones Implementadas

- Verificación de campos vacíos.
- Validación de números en el teléfono.
- Control de índices fuera de rango.
- Verificación de usuarios duplicados.

- Confirmación antes de eliminar pedidos.
-

10. Posibles Modificaciones y Mejoras

10.1 Modificaciones Simples

- Cambiar valores de IVA.
- Modificar precios de los paquetes.
- Ajustar el diseño de la interfaz.

10.2 Mejoras Intermedias

- Guardar datos en archivos o base de datos.
- Mostrar pedidos en tablas.
- Encriptar contraseñas.

10.3 Mejoras Avanzadas

- Implementar Programación Orientada a Objetos.
 - Conectar con una base de datos.
 - Crear reportes de ventas.
-

11. Conclusión

El sistema de Caja Registradora con CRUD y Login en Python cumple con los objetivos planteados, permitiendo aplicar conceptos clave de programación y desarrollo de interfaces gráficas. Además, representa una base sólida para proyectos más complejos en el área de sistemas de gestión.